

Senior DevOps / SRE Interview Q&A (8+ Years Experience Level)

This guide contains realistic interview answers expected from a senior DevOps / SRE / Platform Engineer with 8+ years of experience.

The answers are written in:

- Real interview style
 - Production-focused thinking
 - Structured debugging approach
 - Architecture-level reasoning
 - Leadership-level communication
-

1. Your pod is in CrashLoopBackOff. How do you debug and fix it?

Answer

First, I try to understand whether it's an application issue, configuration issue, or infrastructure/resource issue.

My debugging flow would be:

Step 1 — Check pod status and events

```
kubectl describe pod <pod>
kubectl get events --sort-by=.metadata.creationTimestamp
```

I check:

- Restart reason
- Exit codes
- Probe failures
- Scheduling failures
- OOMKilled events

Step 2 — Check logs

```
kubectl logs <pod>  
kubectl logs <pod> --previous
```

The --previous flag is important because the container may restart too quickly.

Step 3 — Validate dependencies

I verify:

- DB connectivity
- DNS resolution
- Secrets/configmaps
- External APIs
- Service discovery

Step 4 — Resource validation

I check:

```
kubectl top pod
```

Possible issues:

- Memory leaks
- CPU throttling
- Low limits

Step 5 — Compare with healthy pods

If other replicas work:

- Compare node placement
- Compare environment variables
- Compare image versions

Step 6 — Stabilization

If production impact is high:

- Rollback deployment
- Scale healthy replicas
- Disable bad release

Finally, I identify root cause and add prevention:

- Better liveness/readiness probes
 - Resource tuning
 - Alerting
 - Deployment validation
-

2. How do you perform zero-downtime deployments in Kubernetes?

Answer

For stateless applications, I typically use Rolling Updates with:

- readiness probes
- proper maxUnavailable
- PodDisruptionBudgets
- graceful shutdown handling

Example:

```
strategy:
  rollingUpdate:
    maxUnavailable: 0
    maxSurge: 1
```

The application must support:

- SIGTERM handling
- connection draining
- graceful request completion

For critical services, I prefer:

- Blue-Green deployments
- Canary deployments
- Progressive rollout using Argo Rollouts

For stateful applications:

- I upgrade replicas sequentially
- Ensure backward compatibility
- Validate replication health

- Take backups before upgrade

I also monitor:

- error rate
- latency
- saturation
- rollback triggers

A deployment isn't successful just because pods are Running. It's successful only when business SLIs remain healthy.

3. Terraform state got corrupted during backend migration. What would you do?

Answer

First, I immediately stop all further applies to avoid additional corruption.

My recovery process would be:

Step 1 — Protect current state

- Backup local state
- Backup remote backend
- Enable locking

Step 2 — Validate backend integrity

If using S3:

- Check bucket versioning
- Check DynamoDB locks
- Validate IAM access

Step 3 — Pull latest available state

```
terraform state pull
```

Step 4 — Compare infra vs state

I verify:

- Missing resources

- Drift
- Partial applies
- Duplicate resources

Step 5 — Restore or reconstruct

Options:

- Restore previous state version
- Import missing resources
- Remove orphaned state entries

```
terraform import
terraform state rm
```

Step 6 — Validate safely

```
terraform plan
```

I never directly apply immediately after corruption.

Finally, I implement prevention:

- State locking
- Versioned backend
- CI/CD controlled applies
- Separate workspaces
- Backup automation

4. HPA is not scaling even though CPU is high. How do you debug?

Answer

I debug HPA layer by layer.

Step 1 — Validate metrics server

```
kubectl top pod
kubectl top node
```

If metrics are unavailable, HPA cannot scale.

Step 2 — Check HPA status

```
kubectl describe hpa
```

I verify:

- current metrics
- desired replicas
- scaling events
- failure conditions

Step 3 — Validate requests

HPA scales based on resource requests, not limits.

If CPU requests are missing, HPA calculations fail.

Step 4 — Check API server and controller health

Possible issues:

- metrics lag
- API throttling
- controller failures

Step 5 — Validate scaling constraints

I check:

- maxReplicas
- cluster autoscaler
- node capacity
- quotas

Step 6 — Recent deployment/config changes

I compare:

- previous HPA config
- metrics adapter
- Prometheus adapter

Then I stabilize by:

- manual scaling

- fixing metrics pipeline
 - restoring healthy thresholds
-

5. A production pod was OOMKilled but logs are missing. How do you investigate?

Answer

This is a very common real-world issue.

First, I verify whether the container restarted too quickly and logs rotated.

Step 1 — Describe pod

```
kubectl describe pod <pod>
```

I confirm:

- OOMKilled reason
- restart count
- node placement

Step 2 — Check previous logs

```
kubectl logs <pod> --previous
```

Step 3 — Node-level investigation

If logs are gone:

- journalctl
- container runtime logs
- kubelet logs
- dmesg

```
dmesg | grep -i kill
```

Step 4 — Resource analysis

I check:

- memory spikes
- memory leaks
- JVM heap sizing
- cache growth
- traffic spikes

Step 5 — Correlate with monitoring

I correlate:

- Prometheus metrics
- Grafana dashboards
- deployment timelines
- traffic patterns

Step 6 — Prevention

- proper requests/limits
- VPA recommendations
- memory profiling
- heap dumps
- alerting before OOM

The key is not just fixing the crash but preventing repeated instability.

6. Users report 504 errors but the load balancer health checks are green. What is your RCA path?

Answer

This usually indicates partial degradation.

Health checks are often shallow and may not represent real user traffic.

My RCA flow:

Step 1 — Understand impact

- all users or subset?
- single region?

- single endpoint?
- recent deployment?

Step 2 — Validate backend latency

Possible causes:

- slow DB queries
- thread exhaustion
- connection pool exhaustion
- downstream timeout

Step 3 — Compare LB timeout vs app timeout

Common issue:

LB timeout shorter than backend response time.

Step 4 — Check saturation

Golden signals:

- latency
- traffic
- errors
- saturation

Step 5 — Analyze traces

Distributed tracing helps identify:

- slow services
- retry storms
- cascading failures

Step 6 — Mitigation

- scale service
- rollback deployment
- increase pool limits
- cache hot paths
- traffic shaping

Finally:

I update health checks to reflect business readiness, not just HTTP 200.

7. How do you design secure secrets management across 100+ microservices?

Answer

At scale, secret management must be centralized, automated, encrypted, and auditable.

I usually design around:

- Vault or cloud-native secret manager
- short-lived credentials
- RBAC
- workload identity
- secret rotation

My principles:

1. Never hardcode secrets

No secrets in:

- Git
- Docker images
- Helm charts
- Terraform code

2. Use identity-based access

Examples:

- IAM Roles for Service Accounts (IRSA)
- Workload Identity
- Vault Kubernetes auth

3. Automatic rotation

Secrets should rotate automatically:

- DB passwords
- tokens
- certificates

4. Least privilege

Each service gets only required access.

5. Auditability

Every access should be logged.

6. Encryption

- Encryption at rest
- Encryption in transit

7. Runtime injection

I prefer:

- sidecar injection
- CSI drivers
- environment isolation

I avoid long-lived static secrets wherever possible.

8. Explain the difference between Blue-Green and Canary deployments.

Answer

Blue-Green

Two identical environments:

- Blue = current production
- Green = new version

Traffic switches fully after validation.

Advantages

- instant rollback
- low deployment risk
- simple strategy

Disadvantages

- higher infrastructure cost
- full cutover risk

Best Use Cases

- critical production systems
 - major upgrades
 - database-compatible releases
-

Canary Deployment

Traffic shifts gradually.

Example:

- 5%
- 20%
- 50%
- 100%

Advantages

- lower blast radius
- real-user validation
- safer experimentation

Disadvantages

- more operational complexity
- requires observability maturity

Best Use Cases

- high-traffic systems
- microservices
- continuous delivery

In mature environments, I prefer progressive canary deployments with automated rollback based on SLOs.

9. Monitoring shows healthy but users report issues. What could be wrong?

Answer

This is a classic observability gap.

Possible reasons:

1. Metrics are too infrastructure-focused

CPU/memory may look healthy while:

- business transactions fail
- latency spikes
- partial endpoints fail

2. Missing distributed tracing

A downstream dependency may be failing.

3. Alert thresholds are weak

Alerts may not reflect user experience.

4. Synthetic monitoring missing

Internal metrics alone are insufficient.

5. High cardinality masking issues

Aggregation may hide localized failures.

My approach:

I correlate:

- logs
- metrics
- traces
- user reports
- synthetic checks

Then I validate:

- user journey
- regional impact
- dependency health
- browser/network behavior

Good observability should represent customer experience, not just infrastructure status.

10. How do you reduce alert fatigue?

Answer

Alert fatigue usually happens because systems generate noise instead of actionable alerts.

My strategy:

1. Alert on symptoms, not causes

Example:

Instead of:

- CPU > 70%

I prefer:

- user latency affected
- error budget burn

2. Use SLO-based alerting

Alerts should map to business impact.

3. Remove duplicate alerts

I consolidate correlated alerts.

4. Add proper severity

- Critical
- Warning
- Info

5. Add routing rules

Correct team ownership matters.

6. Use inhibition/silencing

Avoid alert storms during outages.

7. Periodic alert review

Every noisy alert must justify its existence.

Good alerts should:

- be actionable
 - reduce MTTR
 - improve reliability
-

11. How do you troubleshoot a service that is reachable internally but not externally?

Answer

I debug layer by layer.

Step 1 — Validate pod health

```
kubectl get pod  
kubectl describe pod
```

Step 2 — Validate service

```
kubectl get svc  
kubectl describe svc
```

Check:

- selector labels
- endpoints
- target ports

Step 3 — Validate ingress/load balancer

```
kubectl get ingress
```

I verify:

- ingress rules
- annotations
- TLS config
- backend mapping

Step 4 — DNS validation

```
nslookup  
curl
```

Step 5 — Network/security validation

- security groups
- NACLs
- firewall rules
- WAF
- network policies

Step 6 — End-to-end testing

I trace:

Client → LB → Ingress → Service → Pod

The goal is to isolate exactly where traffic breaks.

12. How do you handle infrastructure drift in Terraform?

Answer

Infrastructure drift is unavoidable in large organizations unless controlled properly.

My approach:

Prevention

- Restrict manual console changes
- RBAC enforcement
- GitOps model
- CI/CD-only deployments

Detection

Scheduled:

```
terraform plan
```

I also use:

- drift detection pipelines
- cloud config monitoring
- policy-as-code

Remediation

Depending on impact:

- import changes
- revert manual changes
- recreate controlled resources

Governance

I ensure:

- ownership clarity
- tagging standards
- approval workflows

The real solution is operational discipline plus automation.

13. Explain your approach to handling production incidents.

Answer

My incident handling approach has 5 stages.

1. Assess impact

- user impact
- severity
- revenue/business impact
- affected services

2. Stabilize first

Priority is restoring service quickly.

Examples:

- rollback
- scale up
- failover
- disable feature flags

3. Structured investigation

I correlate:

- logs
- metrics
- traces
- deployments
- infra changes

4. Communication

I maintain:

- stakeholder updates
- timelines
- ownership clarity
- incident notes

5. Postmortem

Blameless RCA including:

- trigger
- contributing factors
- recovery timeline
- prevention actions

The goal is not just fixing incidents but improving resilience continuously.

14. What are the Four Golden Signals?

Answer

The Four Golden Signals from SRE are:

1. Latency
2. Traffic
3. Errors

4. Saturation

Latency

Measures response time.

Important to track:

- average latency
- p95
- p99

Traffic

Measures system demand.

Examples:

- requests/sec
- throughput
- queue depth

Errors

Measures failures.

Examples:

- HTTP 5xx
- failed transactions
- timeouts

Saturation

Measures resource exhaustion.

Examples:

- CPU
- memory
- disk
- connection pools

These signals together help detect user-impacting issues early.

15. What happens when a Kubernetes node fails?

Answer

When a node fails:

Step 1 — Node becomes NotReady

Kubernetes detects heartbeat failure.

Step 2 — Pods become unreachable

Pods on that node stop serving traffic.

Step 3 — Service endpoints update

Kubernetes removes unhealthy pod endpoints.

Step 4 — Scheduler reschedules pods

ReplicaSet/Deployment recreates pods on healthy nodes.

Important considerations

Stateful workloads

Need:

- persistent storage
- replication
- graceful failover

DaemonSets

DaemonSet pods restart automatically on new nodes.

Cluster Autoscaler

May provision replacement nodes.

Prevention

I implement:

- multiple AZs
- anti-affinity

- PodDisruptionBudgets
- autoscaling
- node health monitoring

The key is designing systems assuming nodes will fail.

16. How do you secure Kubernetes clusters?

Answer

I follow a layered security approach.

Control Plane Security

- private API endpoint
- RBAC
- MFA
- audit logging

Workload Security

- non-root containers
- read-only filesystem
- seccomp/apparmor
- Pod Security Standards

Network Security

- NetworkPolicies
- service mesh mTLS
- ingress restrictions

Supply Chain Security

- image scanning
- signed artifacts
- trusted registries

Secrets Security

- external secret manager
- encryption at rest
- short-lived credentials

Runtime Security

- Falco

- eBPF monitoring
- anomaly detection

Governance

- OPA/Gatekeeper
- Kyverno
- policy enforcement

Security must exist throughout the entire software lifecycle, not only at deployment time.

17. Why would CPU throttling happen after deploying a logging sidecar?

Answer

This usually indicates resource contention.

Possible causes:

1. Sidecar resource limits too low

The sidecar gets throttled and delays log processing.

2. Shared CPU contention

The main container competes with the sidecar.

3. Excessive log volume

High log generation increases CPU usage.

4. Synchronous logging

Applications blocking on log writes.

My debugging approach

```
kubectl top pod  
kubectl describe pod
```

Then:

- compare before/after deployment
- analyze CPU throttling metrics
- inspect cgroup metrics
- validate log shipping configuration

Fixes

- adjust requests/limits
- async logging
- reduce log verbosity
- dedicated log agents
- batching/compression

Observability tooling itself can become a production bottleneck if not sized correctly.

18. How do you optimize a CI/CD pipeline that takes 20 minutes?

Answer

I first identify where time is spent.

Step 1 — Profile stages

Common bottlenecks:

- dependency download
- Docker builds
- tests
- image scans
- deployment waits

Step 2 — Parallelization

Run independent stages concurrently.

Step 3 — Layer caching

Optimize Dockerfile layers.

Step 4 — Incremental builds

Build only changed services.

Step 5 — Reuse artifacts

Avoid rebuilding identical binaries.

Step 6 — Optimize tests

- parallel tests
- selective tests
- ephemeral test environments

Step 7 — Optimize runners

- autoscaling runners
- pre-warmed agents
- SSD-backed executors

My goal is reducing lead time without compromising quality or security.

19. What is your strategy for observability in distributed systems?

Answer

I design observability around three pillars:

- metrics
- logs
- traces

Metrics

Prometheus + Grafana

For:

- latency
- throughput
- resource usage
- SLO tracking

Logs

ELK or Loki

Structured JSON logs with:

- correlation IDs
- request IDs
- trace IDs

Traces

Jaeger/Tempo/OpenTelemetry

Used for:

- latency breakdown
- dependency mapping
- bottleneck analysis

Additional Principles

- centralized dashboards
- SLO-driven alerting
- sampling strategy
- retention policies
- noise reduction

The objective is reducing MTTR and improving system understanding.

20. What separates a mid-level DevOps engineer from a senior one?

Answer

A mid-level engineer usually focuses on:

- implementation
- deployments
- tooling
- operational tasks

A senior engineer focuses on:

- system reliability

- architecture tradeoffs
- incident leadership
- scalability
- operational maturity
- business impact

The biggest difference is:

A senior engineer thinks in terms of:

- resilience
- failure domains
- blast radius
- automation strategy
- recovery planning
- long-term maintainability

Anyone can deploy Kubernetes. A senior engineer can debug cascading production failures calmly under pressure and design systems that prevent them in the future.